WEEK 2 · DAY 5 · OVERVIEW

Case Study

Boosting on Credit Card Default



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Two weeks of ensembles have been fitted to the card accounts Course 4 modelled: forests, AdaBoost, XGBoost, LightGBM and a stack.

A reviewer asks the desk's questions: which model ranks the defaults highest on accounts it never saw, by how much, is the gap more than noise, and where does the chosen one fail?

The accounts are real, from Taiwan, April to September 2005, so they describe consumer credit there and then, not US credit today.

SEX, **EDUCATION**, **MARRIAGE** and **AGE** are never features: a model using them would sort people by them.

A flag means "flagged at threshold t", never a credit decision about anyone.

OBJECTIVES

Fit the course's models on the training rows, and put a bootstrap interval on a validation score and on the gap between two models.

Tune one booster with a grid inside the training rows, choose the model and its threshold from out-of-fold outputs, report every model on the validation rows, and say what the chosen model leans on and where it fails.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 5

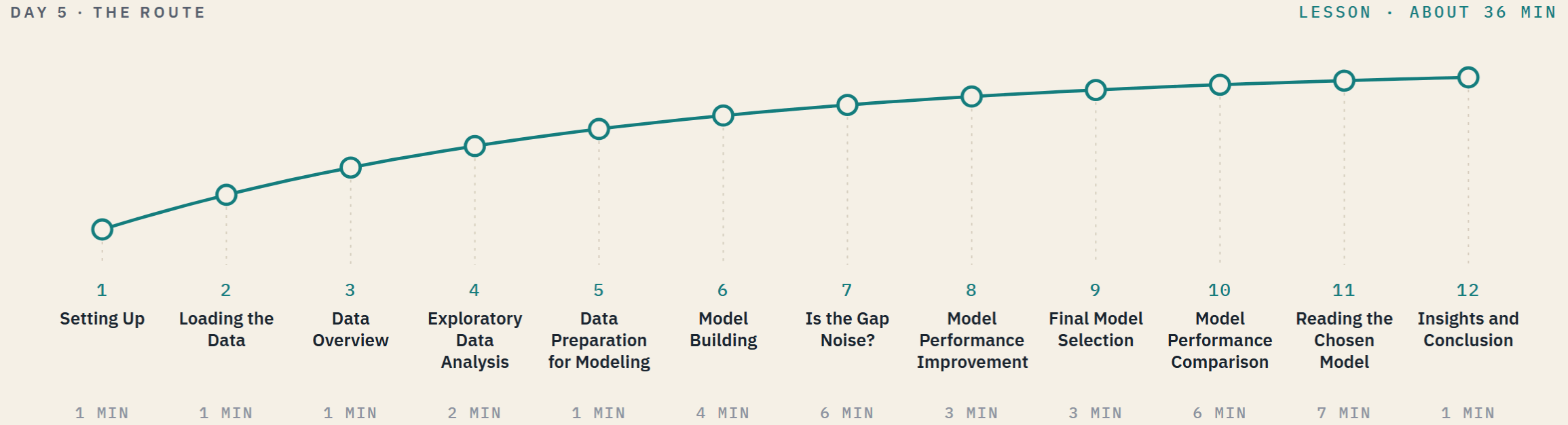


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 36 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
```

THE COURSE'S MODELS

```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
```

TO SPLIT, FOLD, SEARCH AND SCORE

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_predict, GridSearchCV
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import roc_auc_score, average_precision_score, precision_recall_curve, recall_score, precision_score
```

Run this cell and move on: it fixes the seed and finds the course data.

Loading the Data

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
FEATURES = [column for column in card.columns if column not in ["ID", TARGET] + PERSON_COLUMNS]
X, y = card[FEATURES], card[TARGET]
print(len(FEATURES), "features")
```

```
19 features
```

SPLITTING THE ACCOUNTS

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Default share:", round(y_train.mean(), 4), round(y_valid.mean(), 4), round(y_test.mean(), 4))
```

```
Training: 18000  Validation: 6000  Test: 6000
Default share: 0.2212 0.2212 0.2212
```

The same 18,000, 6,000 and 6,000 accounts as Course 4 and every notebook since; the exploration looks at the 18,000 only.

Count the rows, the gaps and the defaults.

Data Overview

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never features

ROWS, GAPS AND DEFAULTS

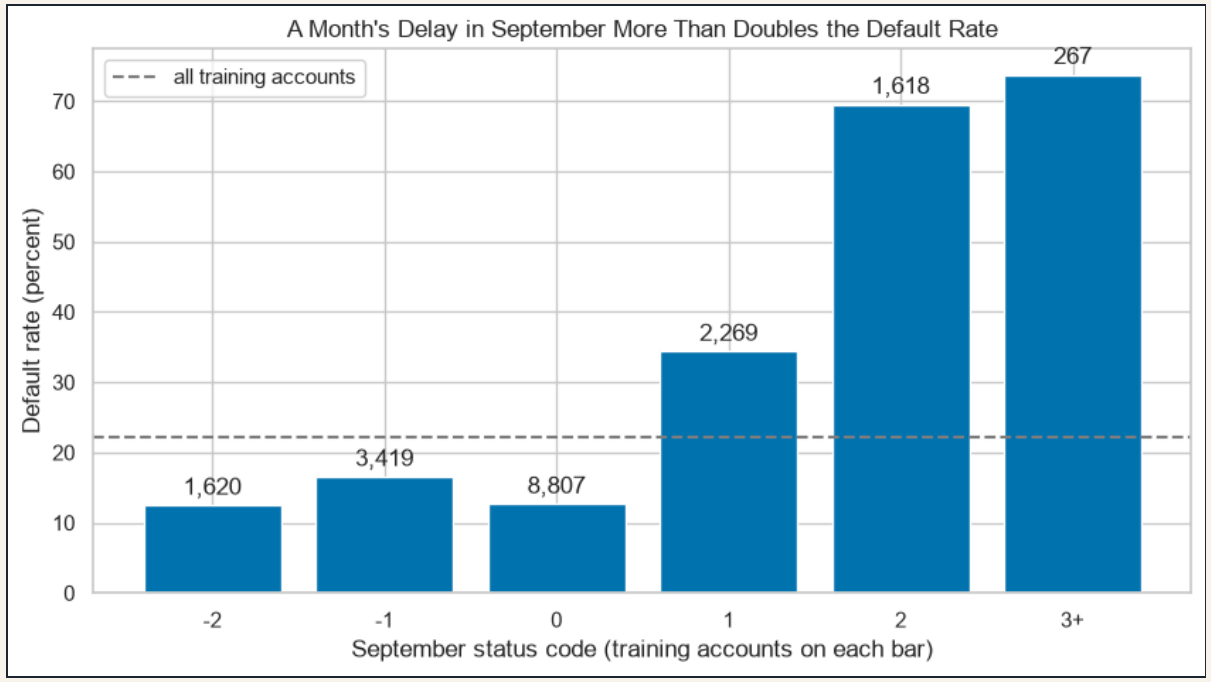
```
print(card.shape, " Missing values:", card.isnull().sum().sum())
print("Defaults:", y.sum(), " Share:", round(y.mean() * 100, 2), "percent")
```

```
(30000, 25)  Missing values: 0
Defaults: 6636  Share: 22.12 percent
```

WHAT THE COUNTS SAY

Nothing is missing, and 22.12 percent of the accounts defaulted.

Exploratory Data Analysis



THE RATE AT EACH CODE

```
(by_status["mean"] * 100).round(2).to_dict()
```

```
{-2: 12.47, -1: 16.53, 0: 12.64, 1: 34.38, 2: 69.53, 3: 73.78}
```

WHAT THE BARS SAY

The three codes 0 and below hold most of the accounts and default at 12 to 17 percent.

One month late in September more than doubles the rate; two months late, seven accounts in ten defaulted.

Trees split on the status codes as they are, so the boosters need no transform; the logistic regression keeps Course 4's scaler.

Fold the training rows five ways.

Data Preparation for Modeling

FIVE FOLDS OF THE TRAINING ROWS

```
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
print("Accounts in each held-out fold:", [len(held_out) for fit_rows, held_out in folds.split(X_train, y_train)])
```

```
Accounts in each held-out fold: [3600, 3600, 3600, 3600, 3600]
```

WHICH ROWS DO WHICH JOB

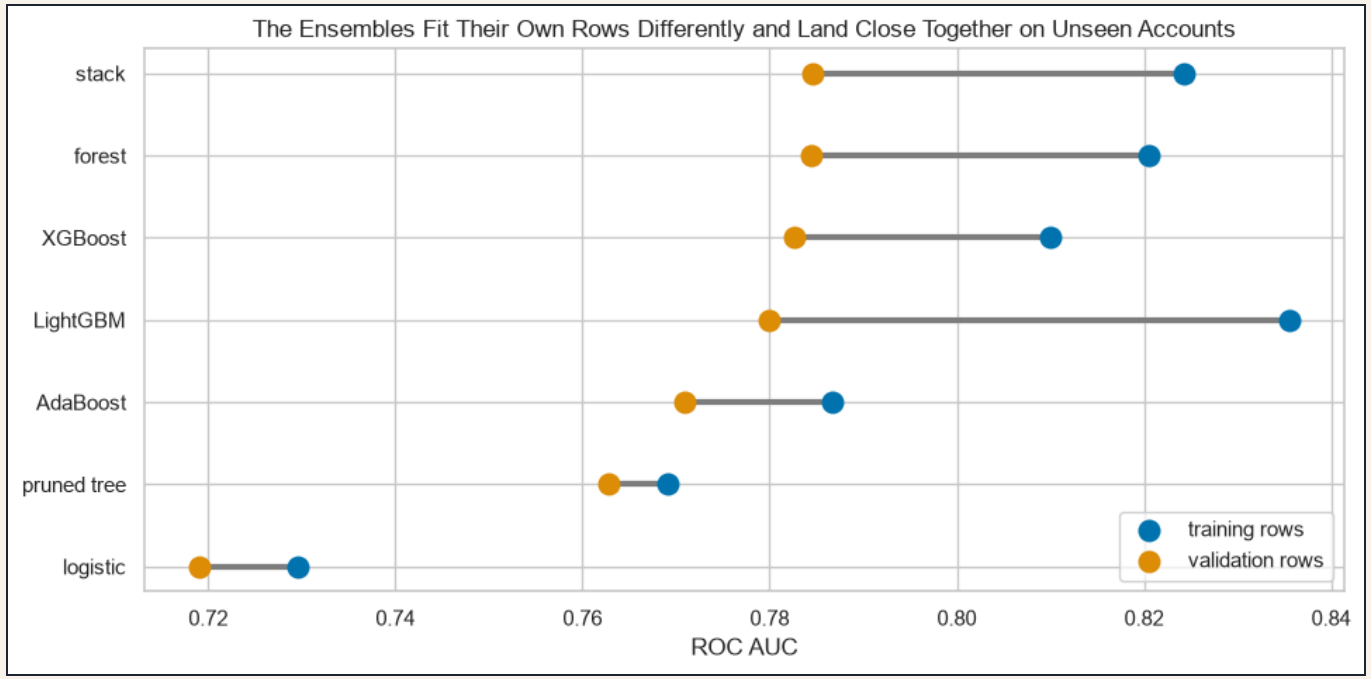
Every model is fitted and every setting and threshold chosen inside the 18,000 training rows.

The validation rows only report, and the 6,000 test rows are never scored in Course 5.

Training rows fit and choose, validation rows report, test rows stay closed.

Fit seven models and score both row sets.

Model Building



VALIDATION ROC AUC

	VALIDATION	AVERAGE PRECISION
stack	0.7846	0.5575
forest	0.7844	0.5562
XGBoost	0.7827	0.5553
LightGBM	0.7799	0.5526
AdaBoost	0.7708	0.5321
pruned tree	0.7627	0.4954
logistic	0.7190	0.4946

THE STACK'S LAYER

```
print("Coefficients:", dict(zip(MEMBERS, layer.coef_[0].round(4).tolist())), " Intercept:", round(layer.intercept_[0], 4))
```

```
Coefficients: {'logistic': -0.6874, 'forest': 1.9285, 'XGBoost': 2.2783, 'LightGBM': 1.7874}  Intercept: -2.6485
```

Is the Gap Noise?

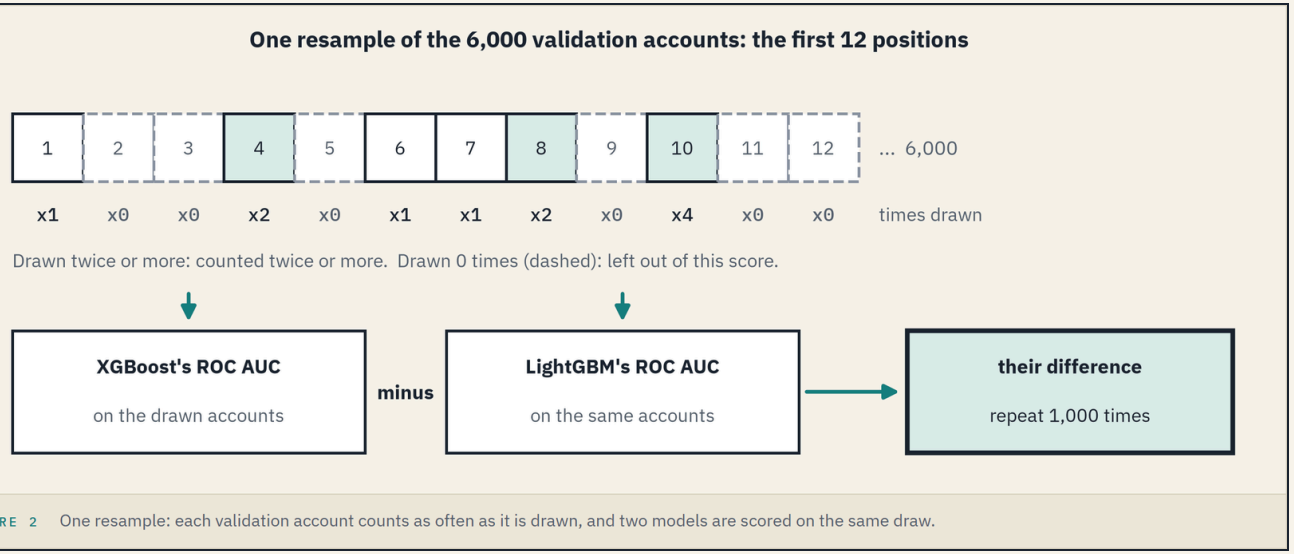


FIGURE 2 One resample: each validation account counts as often as it is drawn, and two models are scored on the same draw.

THE QUESTION

Q. Each model's interval is about 0.028 wide. On the same 1,000 resamples, how wide will the interval of XGBoost's score minus LightGBM's be? Write down a number.

EACH MODEL'S INTERVAL

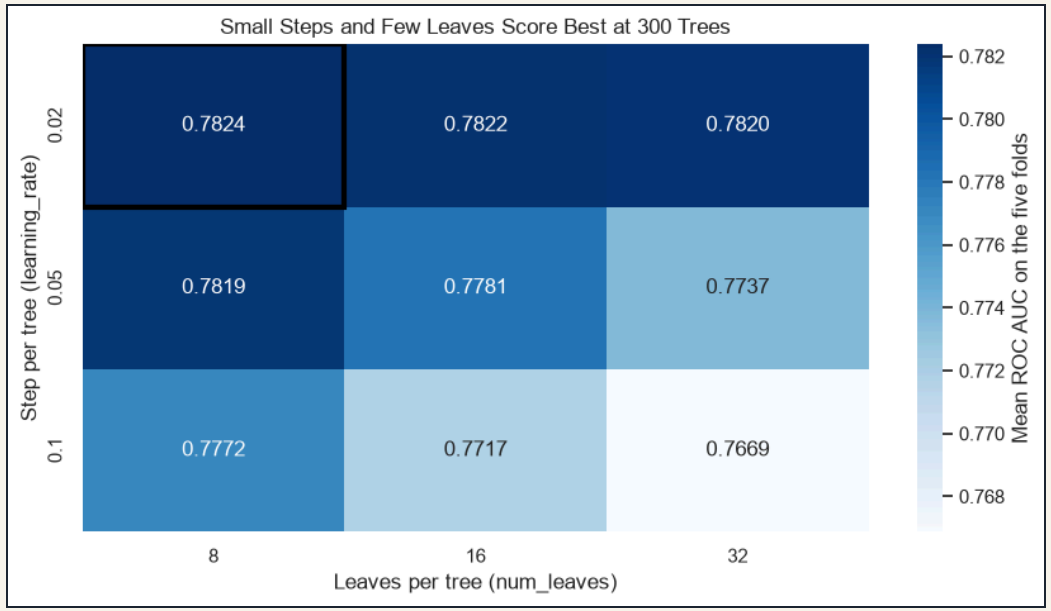
```
XGBoost  interval: 0.768 to 0.7962  width: 0.0282
LightGBM  interval: 0.7653 to 0.7939  width: 0.0286
```

1,000 RESAMPLES OF THE VALIDATION ACCOUNTS

```
rng = np.random.default_rng(SEED)
draws = rng.choice(6000, size=(1000, 6000))
```

Search a small grid of LightGBM settings.

Model Performance Improvement



THE BEST CELL AND THE FOLDS' SPREAD

```
{'learning_rate': 0.02, 'num_leaves': 8}  best_score_: 0.7824
Fold standard deviations: 0.0013 to 0.0034
```

READING THE GRID

The top cells sit within a few ten-thousandths of each other, inside one fold standard deviation.

`GridSearchCV` refits the best cell on all 18,000 training rows: that refit is `search.best_estimator_`.

THE SEARCH

```
grid = {"learning_rate": [0.02, 0.05, 0.1], "num_leaves": [8, 16, 32]}
search = GridSearchCV(LGBMClassifier(n_estimators=300, random_state=SEED, n_jobs=1, verbose=-1), grid, cv=folds,
                      scoring="roc_auc", n_jobs=1)
search.fit(X_train, y_train)
grid_scores = results.pivot(index="param_learning_rate", columns="param_num_leaves", values="mean_test_score")
```

Read thresholds from out-of-fold outputs, then choose.

Final Model Selection

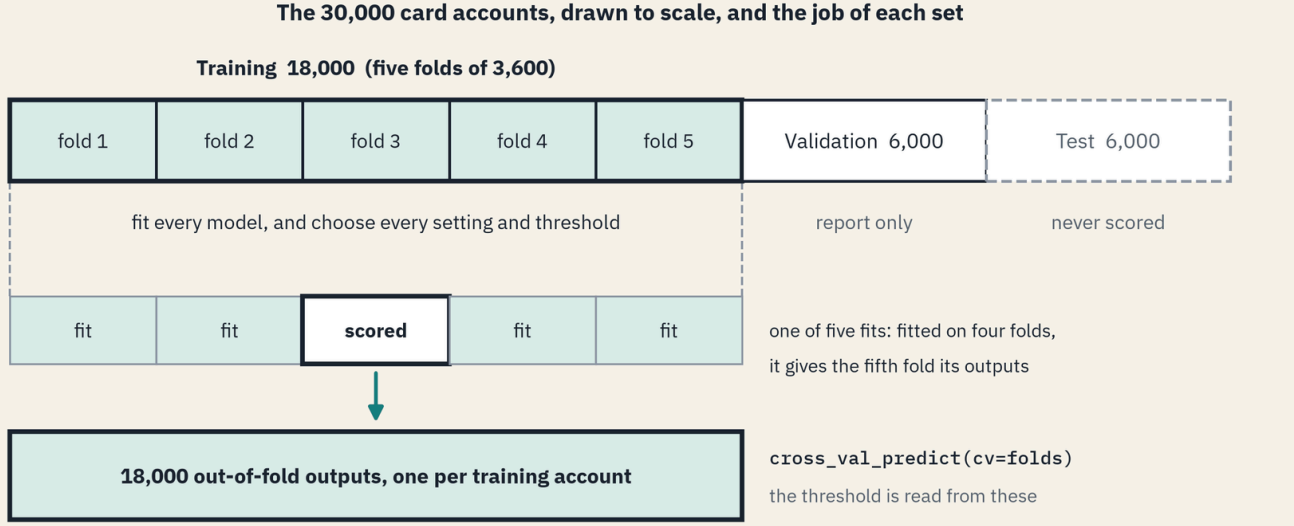


FIGURE 3 The 30,000 accounts by job: five training folds fit and choose, validation only reports, test is never scored.

THRESHOLDS AT A RECALL OF 0.70

	THRESHOLD	FLAGGED SHARE
stack	0.1625	0.3735
XGBoost	0.1895	0.3724
forest	0.1881	0.3728
LightGBM	0.1849	0.3758
tuned LightGBM	0.1844	0.3752
AdaBoost	0.4009	0.3836
pruned tree	0.4015	0.4231
logistic	0.2047	0.4647

THE HIGHEST THRESHOLD THAT REACHES THE TARGET

```
def threshold_for_recall(target, output, target_recall=0.70):
    precision, recall, thresholds = precision_recall_curve(target, output)
    return thresholds[recall[:-1] >= target_recall].max()
```

Report all eight models on the validation rows.

Model Performance Comparison

A QUESTION TO ANSWER FIRST

Q. Each threshold gave a recall of 0.70 on its model's out-of-fold outputs. How many of the eight models reach 0.70 on the validation rows? Write down a count.

ANOTHER QUESTION

Q. Against how many of the other seven models does the stack's paired interval lie wholly above zero? Write down a count.

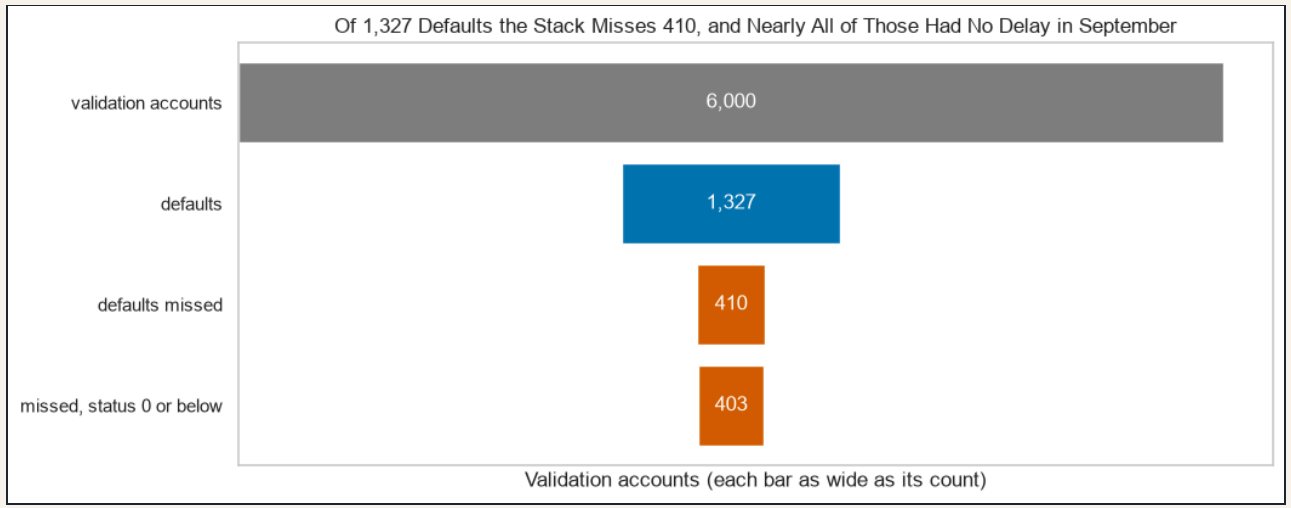
EVERY MODEL ON THE VALIDATION ROWS

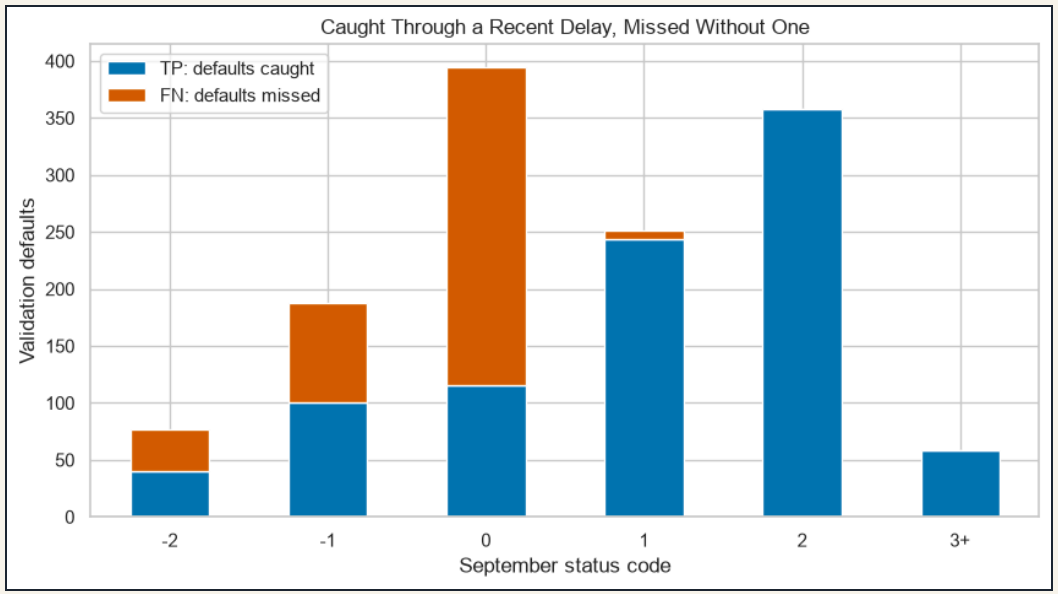
```
report = pd.DataFrame(index=oof.columns)
report["out-of-fold recall"] = [recall_score(y_train, oof[name] >= selection.loc[name, "threshold"]) for name in oof]
report["validation recall"] = [recall_score(y_valid, outputs[name] >= selection.loc[name, "threshold"]) for name in oof]
report["validation precision"] = [precision_score(y_valid, outputs[name] >= selection.loc[name, "threshold"]) for name in oof]
report
```

EVERY GAP WITH THE STACK

```
others = [name for name in order if name != "stack"]
gaps = pd.DataFrame({name: np.quantile(aucs["stack"] - aucs[name], [0.025, 0.5, 0.975]) for name in others},
                    index=["low", "middle", "high"]).T
gaps["clear of zero"] = gaps["low"] > 0
```

Reading the Chosen Model



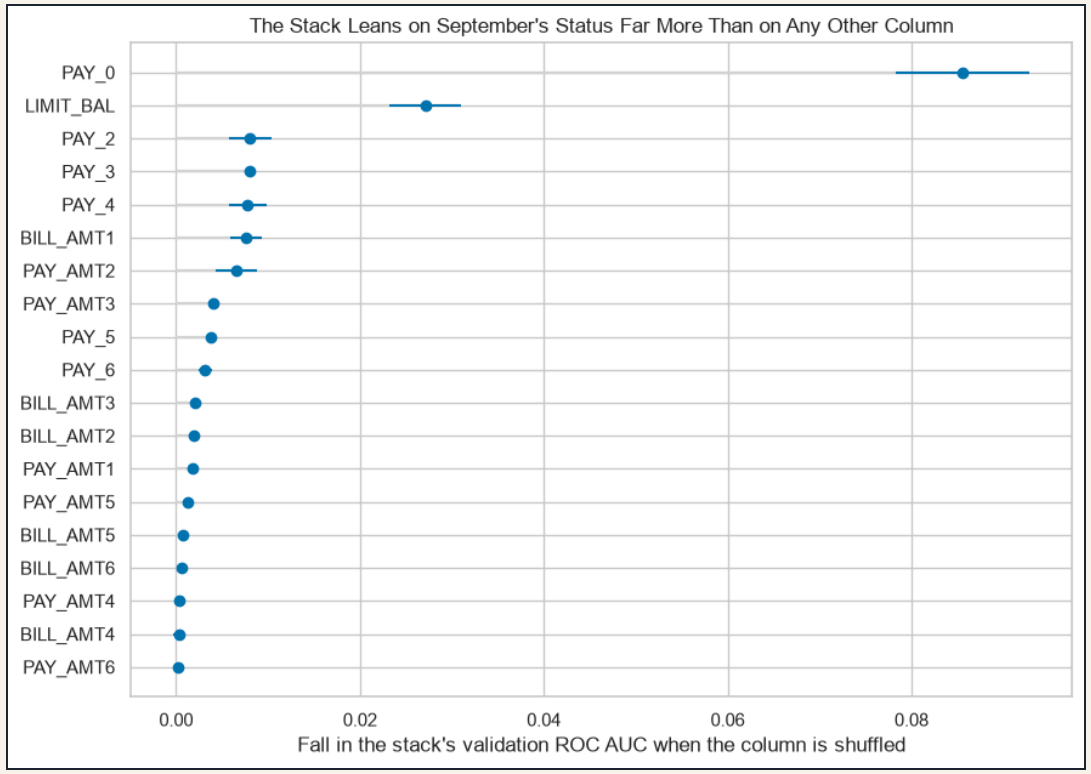


WHERE IT FAILS

```
Flagged: 2198  Defaults caught: 917 of 1327  Missed: 410
Misses at status 0 or below: 403  Paid accounts flagged with a delay of a month or more: 600 of 1281
```

The stack finds defaults through a recent delay and misses the ones that arrive without one, and so do the forest and XGBoost.

Insights and Conclusion



THE THREE COLUMNS IT LEANS ON MOST

	MEAN	STD
PAY_0	0.0856	0.0072
LIMIT_BAL	0.0271	0.0039
PAY_2	0.0081	0.0023

LEFT OPEN

What was not tested: another period, a refit with another seed, any cost of an error, and the test rows.

Where it is wrong: defaults with no delay in September, nearly all of its misses, and the forest and XGBoost miss the same ones.

You Can Now

- Fit and compare many models on the same training and validation rows.
- Put a bootstrap interval on a validation score, and a paired interval on the gap between two models.
- Tune a booster with **`GridSearchCV`** inside the training rows and set its **`best_score_`** beside the validation score.
- Choose a model and a threshold from out-of-fold outputs and check them on rows that only report.
- Rank models resample by resample, and say what a model leans on and where it fails.

NEXT
WEEK 3 · DAY 1

Randomized Search and Honest Tuning

Day 11 measures how far a search's own best score leans.